

JavaScript Variables, Arrays & Objects – Tasks 1 to 15

Ready-to-copy answers for VS Code / Chrome Console

Task 1 – Instagram profile using let

```
let userName = 'Divy';
let followers = 1500;
let isVerified = true;

console.log('userName:', userName);
console.log('followers:', followers);
console.log('isVerified:', isVerified);
```

Creates three let variables and displays all three values in the console.

Task 2 – Instagram profile using let

```
let userName = 'Divy';
let followers = 1500;
let isVerified = true;

console.log('userName:', userName);
console.log('followers:', followers);
console.log('isVerified:', isVerified);
```

Task 2 is the same requirement as Task 1.

Task 3 – Instagram profile

```
let userName = 'Divy';
let followers = 1500;
let isVerified = true;

console.log(userName);
console.log(followers);
console.log(isVerified);
```

Displays the username, follower count, and verification status.

Task 4 – var reassignment and redeclaration

```
var favoriteApp = 'Zomato';
console.log('First value:', favoriteApp);

favoriteApp = 'Swiggy';
console.log('After reassignment:', favoriteApp);

// var also allows redeclaration in the same scope.
var favoriteApp = 'Blinkit';
console.log('After redeclaration:', favoriteApp);
```

var allows both reassignment and redeclaration in the same scope.

Task 5 – var reassignment and redeclaration

```
var favoriteApp = 'Zomato';
console.log('First value:', favoriteApp);

favoriteApp = 'Swiggy';
console.log('After reassignment:', favoriteApp);

// var allows redeclaration in the same scope.
var favoriteApp = 'Blinkit';
console.log('After redeclaration:', favoriteApp);
```

Task 5 is the same requirement as Task 4.

Task 6 – const reassignment

```
const MAX_POSTS = 100;
console.log('MAX_POSTS:', MAX_POSTS);

MAX_POSTS = 200; // This causes an error.
console.log(MAX_POSTS);
```

Expected console error: TypeError: Assignment to constant variable. The script stops at the reassignment line.

Task 7 – const array and push()

```
const favoriteSongs = [
  'Blinding Lights',
  'Perfect',
  'Shape of You'
];

favoriteSongs.push('Believer');

console.log(favoriteSongs);

// This works because const prevents reassignment
// of the array variable, but it does not prevent
// mutation of the array's contents.
```

push() changes the contents of the existing array. The variable still refers to the same array, so no const reassignment occurs.

Task 8 – const array and push()

```
const favoriteSongs = [
  'Blinding Lights',
  'Perfect',
  'Shape of You'
];

favoriteSongs.push('Believer');

console.log(favoriteSongs);

// const prevents reassignment of the variable,
// not mutation of the array contents.
```

Task 8 is the same requirement as Task 7.

Task 9 – spotifyProfile object

```
const spotifyProfile = {
  userName: 'Divy',
  playlists: ['Workout Mix', 'Chill Hits', 'Favorites'],
  isPremium: true
};

console.log(spotifyProfile);
console.log(spotifyProfile.playlists);
```

Creates an object with a string, an array of strings, and a boolean, then logs the complete object and its playlists.

Task 10 – primitive vs reference types

```
let user1 = 'Priya';
let user2 = user1;
```

```

user1 = 'Riya';

console.log('user1:', user1);
console.log('user2:', user2);

let favApps1 = ['Instagram', 'Spotify'];
let favApps2 = favApps1;

favApps1.push('Zomato');

console.log('favApps1:', favApps1);
console.log('favApps2:', favApps2);

// Primitive values are copied independently.
// Arrays are reference types, so both variables
// refer to the same array.

```

After changing user1, user2 remains 'Priya'. After push(), both favApps1 and favApps2 contain Zomato because they reference the same array.

Task 11 – primitive vs reference types

```

let user1 = 'Priya';
let user2 = user1;

user1 = 'Riya';

console.log('user1:', user1);
console.log('user2:', user2);

let favApps1 = ['Instagram', 'Spotify'];
let favApps2 = favApps1;

favApps1.push('Zomato');

console.log('favApps1:', favApps1);
console.log('favApps2:', favApps2);

// Strings are primitive values and are copied.
// Arrays are reference types and share the same
// referenced array.

```

Task 11 is the same requirement as Task 10.

Task 12 – primitive vs object reference

```

let favFood = 'pizza';
let favFoodCopy = favFood;

favFood = 'burger';

console.log('favFood:', favFood);
console.log('favFoodCopy:', favFoodCopy);

let zomatoUser = {
  city: 'Ahmedabad'
};

let zomatoUserCopy = zomatoUser;

zomatoUser.city = 'Mumbai';

console.log('zomatoUser:', zomatoUser);
console.log('zomatoUserCopy:', zomatoUserCopy);

// Primitive strings are copied by value.
// Objects are reference types, so both variables
// refer to the same object.

```

favFoodCopy stays 'pizza'. Both objects show city 'Mumbai' because zomatoUser and zomatoUserCopy refer to the same object.

Task 13 – dynamic typing with typeof

```
function showProfile(name, followers, isPremium) {  
  followers = '1500 followers';  
  console.log('Type 1:', typeof followers);  
  
  followers = 1500;  
  console.log('Type 2:', typeof followers);  
  
  followers = 2500;  
  console.log('Type 3:', typeof followers);  
}  
  
showProfile('Divy', 1000, true);
```

The type of followers changes dynamically: first string, then number, and finally number. JavaScript variables are dynamically typed.

Task 14 – dynamic typing with typeof

```
function showProfile(name, followers, isPremium) {  
  followers = '1000';  
  console.log('Type 1:', typeof followers);  
  
  followers = 2000;  
  console.log('Type 2:', typeof followers);  
  
  followers = 3000;  
  console.log('Type 3:', typeof followers);  
}  
  
showProfile('Divy', 1500, true);
```

Task 14 is the same requirement as Task 13. The typeof operator shows the current type of followers.

Task 15 – displayProfile function

```
function displayProfile(userName, followers) {  
  console.log(`${userName} has ${followers} followers`);  
}  
  
displayProfile('Divy', 1500);
```

Output: Divy has 1500 followers. The function uses a template literal to create the formatted message.